

BROWNFIELD LAB

# Project 3 — Maintaining a Brownfield Repository

The AI is only as good as the context you give it, and you control the context.

## BEFORE YOU START

# Before you start.

 5 MIN

- ✓ Pick your repo: yours, or a fallback (eShopOnWeb for .NET, hexo for Node, LEAP for .NET + React) — start the agent inside that folder.
- ✓ Cut a branch: `git checkout -b brownfield-lab`
- ✓ Own repo → push it: `git push -u origin brownfield-lab`. Fallback → keep the branch local (or fork first).
- ✓ Fallback prereqs: .NET 8 SDK, or Node 22+. LEAP also needs .NET 10 and Docker.
- ✓ Get your feature from the instructor. You will use it twice.

## PART 1 • THE WRONG WAY

# Just build it.

 10 MIN

- ✓ Start the agent inside the repo folder
- ✓ Paste Auto mode on, then Prompt 1 — no planning, no context, default model
- ✓ Let it run. Don't fix anything.
- ✓ Then: did it build? Did it run? Did you see it work?

**PROMPT 1**

**TRANSITION**

# Now do it right.



```
git add -A
```



```
git stash push -m "wrong-way attempt"
```

The AI is only as good as the context you give it, and you control the context.

## STEP 1 · MAP THE CODEBASE

# Map the codebase.

 10 MIN

- ✓ Fresh session, in the repo folder
- ✓ Prompt 2 → save as ARCHITECTURE.md
- ✓ Skim it as it writes. Correct what's wrong — that's context.

Full version: the cartographer plugin — [github.com/kingbootoshi/cartographer](https://github.com/kingbootoshi/cartographer). Run it on your real repo tonight.

**PROMPT 2**

# Get it building and running.

🕒 10–20 MIN

- ✓ Same session
- ✓ Prompt 3 — real output only, no summaries
- ✓ Record the working commands in ARCHITECTURE.md
- ✓ Own project with no way to run it locally? This may be your whole lab, and it'll be worth it. We'll help you find a solution.
- ✓ Agents love to say they're done when they can't prove it. Letting them run the code pays for itself immediately.

## PROMPT 3

## STEP 3 · BUILD THE TEAM

# Create specialized agents (and skills).

 10 MIN

- ✓ Fresh session
- ✓ Prompt 4 → `.claude/agents/`: front-end, back-end, others only if real
- ✓ CLAUDE.md, under 60 lines
- ✓ Delete anything generic

**PROMPT 4**

## STEP 4 • THE RETRO HABIT

# What did we learn? (Ask your robot)

 5 MIN

- ✓ Same session, before you close it
- ✓ Prompt 5 — put it where it belongs
- ✓ Then `/clear`

**PROMPT 5**

# Plan with a stronger model.

🕒 15 MIN

- ✓ Switch to Opus. Enter plan mode.
- ✓ Prompt 6 — tell it your background
- ✓ Orchestrator + Sonnet subagents do; Opus researches
- ✓ Plan → docs/plans/<feature>.md, handoff prompt on top

## PROMPT 6

# Execute the plan.

🕒 20–30 MIN

- ✓ `/clear`, then paste the handoff prompt
- ✓ Prompt 7 — stay on the powerful model, as orchestrator
- ✓ “Not done until it runs locally and I can see it.”
- ✓ Watch how it delegates.

**PROMPT 7**

# Show and tell.

- ✓ Open your feature. Show your neighbors.
- ✓ Compare: `git stash show -p`
- ✓ Same request — what was different?

**The AI is only as good as the context you give it, and you control the context.**

# Bonus topics